EXECUTION-PATH QUALIFICATION AND REALIZED COSTS IN SPECULATIVE DECODING ON CONSUMER SYSTEMS

Anonymous Authors[1]

ABSTRACT

Numerical execution paths can change greedy output, but observing a mismatch does not predict the rest of a trajectory. On the tested B570 stack, we select a draft-free target-only micro-batch-one predictor using exposed discovery data, then freeze complete generated-ID predictions before reserved live speculation. It matches 72/72 capped sequences on 24 repeat-stable prompts, including all 12 runs of four reference-divergent prompts; the same-set reference matches 60/72. The predictor is sufficient for this path, but neither restores qualification nor identifies a unique numerical cause. Separate B570 same-state forks change scores without changing sampled immediate decisions. A new prospective RTX comparison uses all 19 remaining exposure-audited prompts: target-only UB2 matches 51/57 live sequences, whereas proposal-conditioned replay matches 57/57 on shared numerical routines. This rejects the specified target-only candidate, without proving history necessary. On the same prompts, 171 fresh matched cost runs leave batching blocked by direct IDs; serialization qualifies but is slower (0.677× target/serialized latency within an instrumented post-prefill boundary). At fixed progress and target work, 358H cannot reach parity even at zero draft/residual cost. Qualified M4 Guard loses to fixed speculation. These bounded results distinguish predictable trajectories, reference-compatible execution and profitable implementation.

1 INTRODUCTION

Speculative decoding uses cheap proposals to amortize target work (Leviathan et al., 2023). Prior consumer-hardware measurements show both speedups and slowdowns, as well as greedy argmax changes when the target's batch shape changes (Chordiya, 2026). We ask whether an execution-path hypothesis can predict complete, unseen speculative output, including departures from the intended reference.

We test this on one B570 Vulkan stack. A draft-free target-only UB1 predictor is selected after 36/36 discovery matches; its 72 complete ID arrays are then frozen before reserved live outcomes. They match 72/72 runs on 24 repeat-stable prompts, including four reference-divergent prompts (12 runs); the UB128 reference matches 60/72. A wrong later token would falsify the prediction. The result shows that target-only execution suffices to predict this path, without identifying a unique kernel cause. It also gives a clear decision for lossless optimization: this path cannot replace the reference, even when its output can be predicted accurately.

Supporting studies examine intervention and transfer. On a separate B570 cohort, identical fork states yield changed scores but unchanged immediate decisions; some later trajectories differ under a persistent alternate policy. RTX's target-only H1 fails discovery, while independently orchestrated proposal/replay succeeds. A new same-set test then freezes both target-only UB2 and replay predictions before live outcomes on all 19 remaining exposure-audited prompts. UB2 matches 51/57 full arrays; both replay programs match 57/57. This discriminates the predefined candidates, but does not prove proposal/history necessary or a numerical rule portable.

A compatible path still needs a cost test. On those same 19 RTX prompts, a frozen three-condition schedule measures target-only, batched and serialized paths with one timer executable. Batching fails reference equality at 51/57; serialization passes 57/57 but is slower. This joint comparison closes a gap left by the earlier separate qualification and timing campaigns. On 358H, measured target work prevents parity at fixed progress even with draft cost eliminated. On M4, hindsight selection between fixed modes leaves little opportunity, and Guard adds no aggregate benefit. Each case uses its own comparator, so these results do not rank the hardware.

Our three contributions are (1) direct-ID path qualification and matched three-path RTX cost, where compatibility does not bring a speedup; (2) prospective B570 full-trajectory prediction, with separate intervention and a same-set RTX

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

candidate test of its causal and transfer limits; and (3) complete 358H interface accounting and qualified M4 fixed-policy comparisons that show the limits of draft-only and continuation-control improvements.

2 Experimental Question and Measurement Discipline

Acceptance, isolated draft throughput and target-forward batching measure different boundaries. None alone measures the complete speculative loop. We distinguish them using an output contract and complete-cost accounting.

2.1 A path-qualified output contract

Let k be the maximum draft proposal width, UB the target micro-batch capacity (n_{ubatch}), and EOG a vocabulary end-of-generation ID. A greedy speculative round verifies draft proposals, commits the matching prefix and a correction or bonus when budget permits, then removes inconsistent suffix state. Our strict contract requires committed generated IDs to equal the matched target-only greedy reference under the frozen configuration. We can therefore test lossless compatibility with a specified deployed greedy reference for each model, runtime, path and workload. A failed pair blocks a workload-global optimization claim; a small passing probe only establishes eligibility for broader testing. Capped ID equality does not measure functional HumanEval correctness or pass@k.

Historical text-path agreement (H) compares arrays reconstructed from parsed CLI text. Direct-ID qualification (Q) compares captured IDs against the study's frozen reference. Frozen prediction equality (P) compares a sealed prediction with reserved live IDs. A predictor can match a reference-divergent path, so P does not imply Q. Same-state intervention separately compares scores, immediate decisions and subsequent trajectories; these endpoints must not be interchanged.

2.2 Evidence classes

Primary results answer a frozen question after its applicable gate; **secondary** results retain their registered or post-study follow-up scope. **Mechanism-supported** results use path controls or measured regions. **Diagnostic** values describe blocked or restricted paths. Historical RTX controls remain text-path secondary evidence and B570 timing diagnostic, irrespective of later direct-ID results.

2.3 Complete round costs

Let T_b denote matched target-only cost per output token, T_d^{real} the complete realized draft-subsystem time including interface replay, $T_{\text{target,round}}^{\text{real}}$ complete target work, and T_x residual synchronization, transfer and bookkeeping outside those regions. Count all committed output, including correction/bonus, as C. Then

$$S = \frac{T_b C}{T_d^{\text{real}} + T_{\text{target,round}}^{\text{real}} + T_x} \tag{1}$$

$$= \frac{C}{D_{\text{real}} + R_{\text{target}} + X}. \tag{2}$$

Terms in the second denominator are normalized by T_b; break-even requires $C > D_{\text{real}} + R_{\text{target}} + X$. These are accounting identities, not new profitability theory (Kim et al., 2026). Aggregate means and totals require consistent weights; total-time ratios need not equal prompt geometric means. Accepted drafts alone do not count correction/bonus output. Incompatible experiments cannot isolate an unmeasured component by subtraction.

2.4 A separate target-forward proxy

For context n and path e,

$$V_k^{\text{micro}}(n, e) = \frac{T_{\text{target forward}}(k, n, e)}{T_{\text{target decode}}(1, n)}. \tag{3}$$

This narrower microbenchmark omits drafting and may omit correction/bonus and state work; V_k^{micro} is not complete R_{target}. Isolated draft throughput likewise cannot substitute for stateful D_{real}. These measurements help diagnose the tested path but do not identify a universal cause.

3 Experimental Methodology and Platforms

3.1 Frozen systems and workloads

Each study has its own frozen protocol. Table 1 lists questions, comparators and sample units. Appendix K gives historical settings; Appendix Q specifies RTX V4. RTX's original direct-ID OS build is 26200; follow-ups use 26300. Driver versions alone do not identify driver bytes. Final comparisons process one request at a time. The target/draft pair is Qwen3-8B/0.6B Q8_0, except the 358H adapter's symmetric INT4 OpenVINO draft.

The representative M4 study selects 24 unseen Spec-Bench prompts across six groups (Xia et al., 2024). A separate preregistration selects 30 HumanEval tasks before input inspection (Chen et al., 2021); fixed $k = 2$ is primary and threshold variants secondary. Three repetitions and a 128-ID cap apply to the code studies. The prospective runtime studies retain their separate task splits and estimator in Section 7.

For direct-ID studies, "complete" means all committed/generated IDs through the first inclusive EOG or 128-ID cap, excluding prompt IDs and uncommitted proposals.

Table 1. Study map. Counts are distinct prompts × repetitions; complete ID endpoints include the first EOG or cap128. H = historical text-path comparison, Q = reference qualification, P = frozen prediction. Timing is matched only within the named study. M4 uses Metal, RTX CUDA, B570 Vulkan; 358H combines Vulkan target and OpenVINO draft. Details and evidence entries are in the separate appendix.

Question / study	Reference or contrast	Prompts × reps	Endpoint / role	Matched timing
Historical costs	M4, RTX, B570 own baselines	24/30 × 3	Parsed text (H)	Historical only
Direct-ID gates	RTX UB1; B570 UB128	30 × 3 each	All IDs (Q)	None
B570 H1 prediction	UB1 predictor; UB128 reference	24 × 3	72/72 P; Q blocked 60/72	None
B570 live fork	Same state UB1 vs UB128	New 24 × 3	Scores, decisions, later IDs	None
RTX H1 discovery	Target-only UB2 vs live	12 × 3	24/36; reserved 0	None
RTX V2 P/R replay	Sealed proposals; UB1 reference	New 24 × 3	72/72 P; Q blocked 60/72	None
Prior RTX serialized cost	Own target-only UB1	30 × 3	90/90 Q; latency	Instrumented CLI
RTX V4 discriminator	UB2 vs sealed replay vs live	New 19 × 3	51/57 vs 57/57 P	None
RTX V4 cost	Own UB1; batched/serialized	Same 19 × 3	51/57 blocked; 57/57 Q	Three-path CLI
358H Phase 3	Own target-only; CPU/NPU draft	30 × 3	90/90 Q per path; cost	Complete loop
M4 runtime control	Target, Always; fixed-mode oracle	60 × 3	180/180 Q per policy	Charged loop

B570 H1 arrays all reach the cap; RTX also has earlier EOGs. Audit and deterministic selection precede reserved execution, and predictions/tapes are sealed before reserved live. RTX V4 uses all 19 tasks remaining after 145 exposure exclusions, including seven exposed by the failed V3 attempt. A freezes four sets of 57 pre-live arrays, then executes 57 native live runs; B measures 171 fresh runs on the same tasks. These are 19 distinct prompts, not 456 independent samples. The cohort is exposure-conditioned, not a representative random sample. No-known-exposure covers only audited records. Appendix Q retains failures, the disclosed protocol amendment and endpoint rules.

Timing excludes loading and initial prefill. Native historical counters and the 358H complete-loop timer keep their original boundaries. The new RTX timer starts after synchronized target/draft prefill and before per-request sampler and generation-state work, ending at slot release; its instrumented CLI boundary is detailed in Section 5.4. Different runtimes, quantization and references preclude hardware-only causal comparisons.

3.2 Statistical estimators and uncertainty

Historical M4/RTX/B570 compute the GM of three paired ratios for each prompt; 358H uses their median. Headline speedups are prompt GMs with 95% prompt-bootstrap intervals. M4 HumanEval uses 100,000 resamples, the other hardware campaigns 10,000. Both later RTX cost studies use each condition's three-time median, then reference/candidate ratio and prompt GM. V4 uses 10,000 paired-prompt draws, seed 2026100204, frozen task order and linear percentiles. Candidate cost interpretation requires every reference-ID pair and stable reference repeats; blocked-path timing is diagnostic. M4 runtime retains its own streams. Repetitions and events are correlated. Intervals exclude full session, thermal and recalibration uncertainty.

3.3 Provenance and energy boundaries

An evidence ledger maps each study's canonical bytes and freeze roles. Hashes identify content; server acknowledgements and host records supply reported ordering, not trusted external process time. Anonymous derivatives preserve arrays but cannot authenticate original absolute chronology or omitted weights/runtime/compiler/driver bytes. Appendix O lists exact packages and capabilities; offline checks execute no models.

M4 energy measurements cover the package aggregate and exclude separate DRAM; B570 measurements cover the adapter board. RTX mixed domains are not a common system boundary. No end-to-end 358H Phase-3 or runtime energy result is claimed. Historical Windows per-

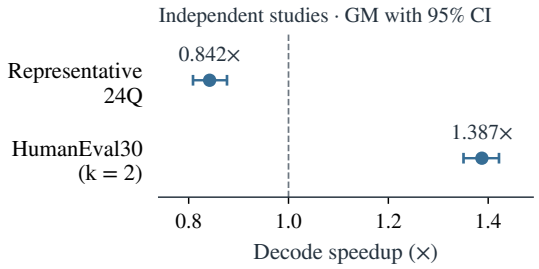


Figure 1. **Opposite profitability regimes on the same tested M4 stack**. Representative 24Q yields 0.842× [0.809, 0.877], while the separately preregistered HumanEval30 fixed-$k = 2$ study yields 1.387× [1.351, 1.422]. Points are campaign prompt geometric means of paired decode speedups; intervals are campaign-specific 95% prompt-bootstrap intervals. The dashed line marks parity. Both pass the recorded re-tokenized-text comparator (72/72 and 90/90 pairs); these are not direct generated-ID comparisons or functional HumanEval pass@k. These are separate studies, not a paired between-workload comparison; differing workloads and session conditions do not isolate workload-only causality.

run provenance is weaker; 358H source reconstruction is strongly supported by retained hashes and code behavior, but exact historical identity is unproven (the internal `STRONGLY_SUPPORTED` label). No platform energy ranking follows.

Existing-record analysis. We calculate the score distributions, fixed-mode selection opportunity and conditional cost bounds after the study, with aggregate outcomes already known. A separate plan fixes inputs, ordering and a new sensitivity seed before recomputation; no original endpoint or confidence interval is replaced. Appendix P gives the formulas and identifies the anonymous inputs.

4 Gains Do Not Translate Uniformly

Table 2 retains historical text-path timing for M4/RTX/B570. The separate direct-ID gates in Section 5.1 and Figure 3 do not certify those timings. Later RTX cost studies qualify their own instrumented paths; the new matched three-condition result is in Table 4. A blocked path remains diagnostic.

4.1 M4: outcomes in two separate studies

The representative frozen M4 sample produces 0.842× speedup (95% CI [0.809, 0.877]), with 72/72 text-path-equal pairs and only 1/24 prompts faster. The separately preregistered HumanEval fixed-$k = 2$ study produces 1.387× [1.351, 1.422], with 90/90 text-path-equal pairs and 30/30 prompts faster. Figure 1 retains both; the positive code-conditioned result does not replace or pool with the negative representative result.

The two studies observe opposite outcomes on the same hardware, with different workloads and sessions. These are not workload-only causal effects or general M4 profitability. The discovery effect size is not a confirmatory estimate; early MLX failures do not negate the historical Q8_0 Metal result.

For primary fixed-$k = 2$ HumanEval, the package-efficiency ratio (baseline/speculative J per output token) is 1.013× [0.990, 1.034]. The interval includes parity; a package-energy improvement is not established. The registered $p_{\min} = 0.9$ condition has secondary energy evidence, discussed separately in the appendix. The preregistered corrected synthetic ceiling was not executed. We retain that deviation rather than treating the earlier synthetic diagnostic as its replacement.

4.2 Across stacks: negative and blocked outcomes

On RTX 5070 Ti Laptop, the serialized CPU-to-RTX control reaches 0.404× [0.390, 0.418], while the same-GPU draft control reaches 0.650× [0.629, 0.673]. Both pass 90/90 text-path comparisons and accelerate zero of 30 prompts. The higher speed of same-GPU placement shows that placement and associated integration costs matter, but its continued loss rules out transfer as the sole explanation.

On Arc B570, neither the fast path nor the serialized fallback supports a workload-global admissible optimization result. The latter's diagnostic speedup is 0.3087× [0.2927, 0.3246], with zero of 30 prompts faster. That interval describes timing on a partially mismatching path; its precision does not turn it into an admissible speed claim. The formal failure and stop rule remain the platform result. Board-energy observations likewise remain diagnostic and boundary-limited.

On Core Ultra X7 358H, the closed Phase-3 heterogeneous runner has direct-ID sequence equality on both CPU and NPU drafting: 90/90 pairs per path, with zero of 30 prompts faster. CPU achieves 0.734× [0.693, 0.771]; NPU achieves 0.352× [0.333, 0.369]. Both qualified paths are slower despite favorable isolated draft measurements. Section 5 examines the measured costs behind this slowdown.

5 From Local Opportunity to Realized Gain

We test output qualification, frozen path prediction and sampled intervention effects, then examine realized cost on a qualified path. Figure 2 shows the original B570 discovery, freeze and reserved test; later studies use their own tasks and gates.

Table 2. Decode cost by evidence group. All speedups are baseline/speculative latency (×; higher is faster), with frozen prompt estimators and 95% CIs. Historical agreement does not qualify direct IDs. RTX V4 matched cost is in Table 4; original Figure 3 gates have no matched latency.

Path / workload	Speedup [95% CI]	Endpoint	Pairs	Faster prompts	Class
Historical text-path timing					
M4 representative 24Q	0.842 [0.809, 0.877]	Text-path	72/72	1/24	Primary
M4 HumanEval fixed-$k = 2$	1.387 [1.351, 1.422]	Text-path	90/90	30/30	Primary
CPU to RTX 5070 Ti Laptop	0.404 [0.390, 0.418]	Text-path	90/90	0/30	Secondary
RTX 5070 Ti Laptop to itself	0.650 [0.629, 0.673]	Text-path	90/90	0/30	Secondary
B570 serialized fallback	0.3087 [0.2927, 0.3246]	Text-path	60/90	0/30	Diagnostic
358H direct-ID-qualified complete-loop timing					
358H CPU draft	0.734 [0.693, 0.771]	Direct-ID	90/90	0/30	Primary
358H NPU draft	0.352 [0.333, 0.369]	Direct-ID	90/90	0/30	Primary

Stage	Design or comparator	Complete generated-ID endpoint
Discovery	12 prompts × 3 runs; select target-only UB1 as H1.	UB1 predicts the serialized trajectory in **36/36** runs.
Hypothesis freeze	After discovery; fix H1, reference, tasks and endpoint.	Before reserved reference/predictor inference; H2 remains unrun.
Prediction freeze	After pre-live inference; seal all 72 UB1 arrays.	Before the first reserved live outcome; no prediction edits.
Live test	Compare each of 72 live serialized runs with both frozen paths.	UB1 predictor/live **72/72**; UB128 reference/live **60/72 (BLOCKED)**. Four prompts (12 runs) are reference-divergent.
Divergent case	HumanEval/134; first difference at generated-ID index 88.	Reference ID 914 → predictor/live ID 1946 in all 3 runs.

Figure 2. **B570 prospective prediction.** Hypothesis and full-array freezes are separate stages. The 24 prompts × three repeat-stable runs test complete capped trajectories, including four non-reference predictions. Reported ordering and retained arrays are reviewed in the appendix; prediction success does not restore qualification.

5.1 Execution-path qualification

Figure 3A retains eleven historical target-forward proxy cells. Metal $V_2^{\text{micro}} \approx 1.02$–$1.08$, near-step RTX batching, and B570 fast 1.24–1.64 versus serialized 1.983–1.992 describe different runs and contexts. Lower proxy values mean cheaper target-forward work relative to one-token decode. These measurements omit integration, cannot replace complete R_{target}, and have no same-study matched cost/qualification tradeoff with Panel B.

Original RTX and B570 direct-ID campaigns each use 30 prompts × three, with no timing and a 90/90 gate. RTX matches 75/90 batched and 90/90 serialized; only serialization qualifies. B570 matches 60/90 fast and 78/90 serialized; both are blocked. Historical and fresh B570 60/90 counts are different comparisons. Repeat stability does not imply reference equality; Figure 3A/B cannot form a matched speed/qualification tradeoff. RTX V4 below measures a new matched study, without certifying historical timing.

5.2 B570 prediction and sampled live intervention

A target-only harness first reproduces 90/90 UB128 reference sequences. H1 target-only UB1 matches 36/36 sequences on 12 exposed discovery tasks; its frozen tree selects H1, leaving H2 unrun. After conservative exposure exclusion, 24 reserved tasks are selected. H1 freezes before reserved inference and all 72 full predictions freeze before reserved live. Prediction/live agrees 72/72 over three stable runs per prompt, including four reference-divergent prompts (12 runs); the same-set UB128 reference agrees only 60/72 and remains *BLOCKED*. All these reserved arrays reach 128 IDs. The predictor could have failed on any reserved sequence; its success establishes predictive path sufficiency on this B570 stack, not a unique numerical cause. The earlier serialized 78/90 gate has a different task set and reference comparison.

A separately frozen fork study uses 24 new prompts × three, sampling each first verification block after at least 64 committed IDs. UB1/UB128 branches share checked KV, prefix, positions, proposals and cloned sampler state. Scores change at 72/72 forks, but raw top-one and actual immediate

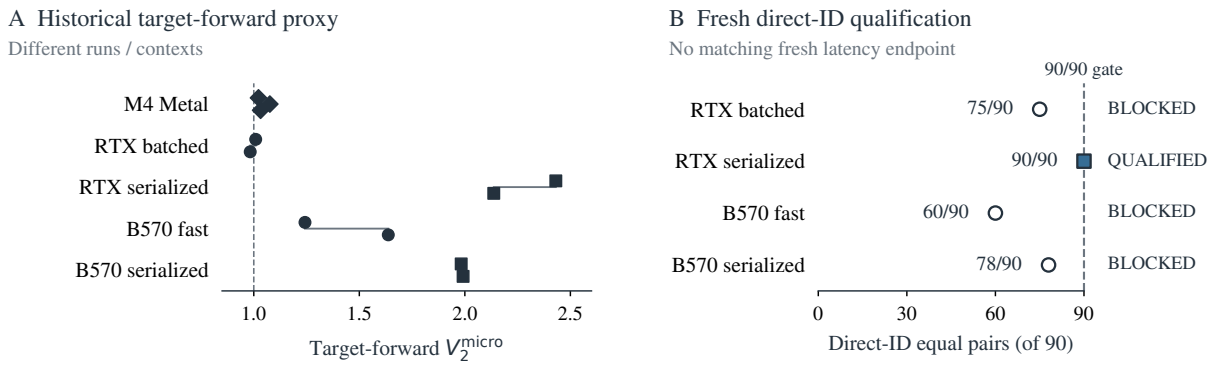


Figure 3. **Historical target-forward proxy and fresh direct-ID qualification are separate evidence.** A: historical V_2^{micro} cells from different runs and contexts; shapes encode M4 reference (diamond), fast/batched (circle), and serialized (square) paths, with no qualification encoding. Segments span measured contexts, not uncertainty. The proxy is not complete R_{target} or end-to-end speedup. B: fresh HumanEval30 direct-ID sequence equality, with no matching fresh latency endpoint; only RTX serialization reaches the frozen 90/90 gate. Open circles mean BLOCKED; the filled square means QUALIFIED. The panels are not a matched cost–qualification experiment. Fresh direct-ID qualification does not retrospectively authorize historical timing. Results are path/runtime scoped; contexts and references differ.

decisions change at 0/72. Later complete trajectories differ in 18/72 runs on 6/24 prompts. Repeats and within-block events are correlated.

The alternate branch restores draft state and continues with UB128 target blocks; later proposals follow its own live history. Delayed differences therefore cannot be attributed to the sampled block alone or treated as immediate reference restoration. Existing-record analysis of 216 correlated raw-logit rows gives median absolute top-one score change 0.0695; top-one IDs remain unchanged. The six sensitive prompts first diverge 10–50 positions after the fork. Appendix P retains all score/margin summaries, positions and clone/R0 controls. These findings identify no unique operation or whole-generation null effect.

5.3 RTX same-set trajectory discriminator

Earlier RTX H1 target-only UB2 fails discovery at 24/36 and executes no reserved runs. Separate V2 H2/live, R_ONLY/live and H2/R_ONLY match 72/72, with UB1 reference/live 60/72. H2's draft-only P and target-only R processes compute their own feedback and decisions; fresh R_ONLY reads the sealed proposal tape sequentially, while native live imports no tape. This is application/API orchestration independence on shared llama.cpp/GGML/CUDA routines. V2 lacked a same-set target-only competitor.

V4 fills that gap on all 19 remaining exposure-audited tasks. Four pre-live conditions—UB1 reference, target-only UB2, H2 and fresh R_ONLY—produce 228 complete arrays; all predictions and tapes are frozen and remotely retrieved before 57 native batched live runs. Table 3 contrasts the predefined candidates. UB2/live matches only 51/57 on 17/19 prompts; H2/live, R_ONLY/live and H2/R_ONLY each match 57/57. Reference/live is 51/57, including two reference-divergent prompts that both replay programs predict. All five conditions are repeat-stable.

Table 3. RTX V4 prospective full-array prediction against native batched live. All rows use the same 19 prompts × three; predictions precede live. These are prediction comparisons, not reference qualification.

Predictor / comparator	Pairs / 57	Prompts / 19
Target-only UB2	51	17
Independent P/R (H2)	57	19
Fresh R_ONLY	57	19
UB1 reference	51	17

Equal aggregate counts do not make UB2 and reference the same predictor: UB2 fails on /15 and /25, reference on /15 and /18. The test rejects this specified UB2 candidate's full-cohort sufficiency. It does not reject every target-only hypothesis or prove proposal/history necessary, minimal or unique. Replay uses drafting to construct its tape and shared numerical code to execute it. Appendix Q gives first-divergence IDs, endpoint distributions and the failed V3 capture history; V3 contributes no observations here.

5.4 RTX matched qualification and cost

The earlier two-condition cost study freshly qualifies serialization at 90/90 and measures target/serialized 0.646×

Table 4. RTX V4 matched qualification–cost: 171 fresh runs on 19 prompts × three × three paths. Target/serialized use UB1, batched UB2; speculation uses $k = 2$. Speedup = target/candidate latency. Blocked timing is diagnostic; no speedup or CI is formed. Serialized is faster on 0/19 prompts.

Path	ID gate	Speedup [95% CI]
Target	Reference	1.000
Batched	51/57 blocked	—
Serialized	57/57 qualified	0.677 [0.662, 0.692]

[0.630, 0.660] on 30 prompts (180 runs). It contains no batched condition and remains a separate historical result.

V4 B uses the same 19 tasks as A but 171 fresh observations: target-only UB1, batched $k = 2$ UB2 and serialized $k = 2$ UB1. One frozen timer executable, model/runtime, input, endpoint and balanced order schedule cover all three paths. Each condition occupies each order position 19 times. Native micro-batch configuration also propagates to draft; this is a whole-path comparison, not a target-only causal intervention.

The fresh ID gate leaves batching *BLOCKED* at 51/57. Serialization passes 57/57; its prompt-median target/candidate GM is 0.677× [0.662, 0.692], with 0/19 prompts faster. Its reciprocal relative latency is 1.476×. Blocked batched measurements remain diagnostic and supply no speedup/CI. The design jointly tests reference compatibility and qualified cost without borrowing a gate from another campaign; it establishes no fast admissible path. The new 0.677 and earlier 0.646 estimate use different cohorts and are not an optimization trend.

The native timer charges per-request sampler/state work after synchronized prefill, drafting, target validation, acceptance/correction/bonus, KV rollback, synchronization and related logging/response through slot release. Loading, initial prefill and teardown are excluded. Observer controls check output stability on engineering inputs; clock probes bound clock-call overhead only. Condition-dependent debug logging need not cancel. The result is instrumented post-prefill request latency, not production throughput, uninstrumented deployment or energy. Appendix Q specifies the estimator, frozen paired-prompt bootstrap and raw endpoints.

5.5 Stateful realization

On 358H, isolated NPU draft throughput exceeds CPU at tested contexts, but both integrated direct-ID-qualified paths are slower than target-only. Complete committed progress is $C = 2.4631$ and 2.4584, including correction/bonus. Complete target work consumes 87.34% of CPU-path time. The NPU synchronous OpenVINO region combines full-prefix replay/reconstruction and proposal, consuming 58.35%; these costs are inseparable. Residual orchestration is 0.04% and 0.02%.

Figure 4 describes the measured interface, not NPU capability or a measured persistence benefit. Arithmetic totals give CPU $(D_{\text{real}}, R_{\text{target}}, X) = (0.4304, 2.9767, 0.00124)$ and NPU $(4.1251, 2.9435, 0.00130)$, yielding 0.7227× and 0.3477× in Equation 2; these are total-time ratios rather than Table 2's prompt GMs. With measured progress, target-work and baseline totals fixed, nonnegative draft and residual terms imply $S \le C/R_{\text{target}}$: 0.8275× CPU and 0.8352× NPU, even if both other terms vanish. Thus draft-only savings cannot reach parity under this accounting. This post-study arithmetic bound is not a prompt-GM bound or an optimization experiment; changing progress or target execution invalidates its fixed assumptions. The later persistent-state/wider-target candidate stops at qualification without formal performance or energy.

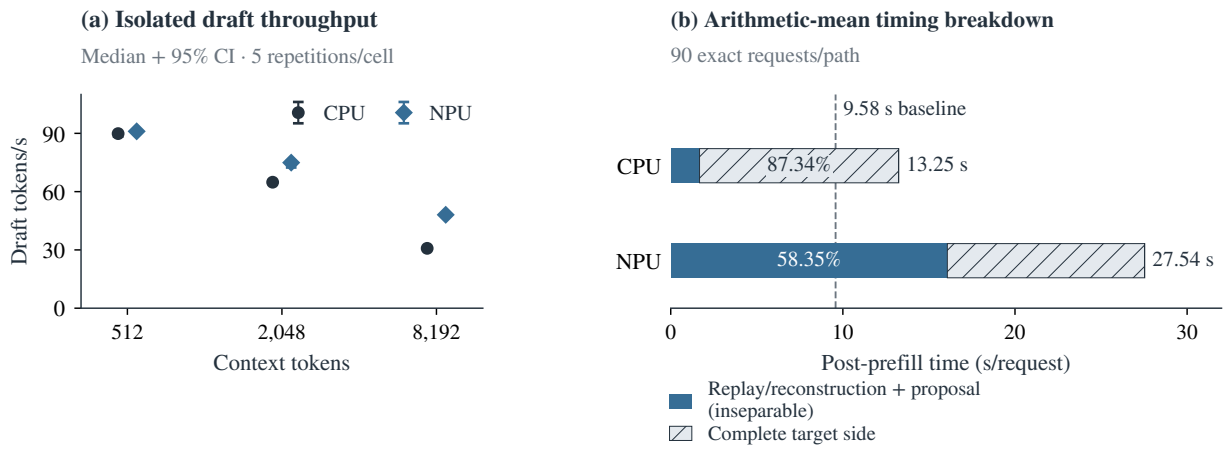
6 From Measured Constraints to Runtime Decisions

Batch-shape sensitivity is already known. B570 adds a prospective test that could fail on any reserved ID. Four correctly predicted non-reference trajectories show that the predictor does more than echo the reference; the path must still be rejected as a lossless replacement. Unchanged sampled fork decisions limit the causal explanation. RTX V4 tests the missing same-set competitor: the specified UB2 predictor fails while replay agrees throughout. This rejects that candidate's full-cohort sufficiency, not all target-only predictors, and does not establish a necessary or minimal history state.

The matched RTX schedule gives two distinct decisions: exclude batched execution as a reference-compatible replacement, and reject measured serialization as a latency optimization within this instrumented boundary. It establishes no fast qualified alternative. On 358H, eliminating draft and residual cost cannot reach parity at fixed progress and target work; improving those terms alone is insufficient. The M4 study below tests whether continuation control improves a qualified fixed path. These decisions use each study's comparator and establish neither a common numerical cause nor a performance law across stacks.

7 Qualification and Realized-Cost Runtime Control

We examine qualification and continuation in separately frozen runtime studies. The 358H candidate stops at direct-ID qualification. On M4, a prospectively direct-ID-qualified path permits a test of a simple cost rule: Guard remains faster than target-only, but fixed speculation is faster in the



Separate prompt-GM speedup: CPU 0.734×, NPU 0.352×. Residuals <0.05%; true widths retained.

Figure 4. **Isolated draft advantage and complete 358H cost.** A: five-repeat steady-state draft throughput with repetition-bootstrap intervals. B: arithmetic mean post-prefill seconds over 90 exact observations per path, against the shared baseline. NPU's 58.35% region combines full-prefix replay/reconstruction and proposal; it is inseparable. CPU complete target work is 87.34%. These bars differ from the prompt-GM speedups in Table 2; persistence savings are unmeasured.

post-study paired secondary comparison; no incremental control benefit is shown. The studies test the decisions in Section 6; neither establishes a successful optimization intervention.

7.1 An intervention stopped by qualification

The 358H component measurements motivated persistent draft state and a wider target execution setting. The implementation ran the target and draft backends in separate processes. Synchronous draft calls, including communication and state work, remained inside the target worker's post-prefill timer. The implementation then had to pass correctness qualification. CPU persistent-state restore/reconcile correctness could not be established under the tested representation; the Optimum NPU state-capable path was unavailable with the frozen dynamic-shape IR. These are implementation boundaries, not evidence that the devices cannot retain state.

The target setting `n_ubatch=2` (UB2) passed the small gate but failed full qualification. Correctness-only follow-up controls on Qwen3-8B Q8_0, llama.cpp b10839, and Vulkan0 reproduced different greedy tokens under UB1 and UB2 even with identical exact prefix tokens. Request-order and separate-process controls reproduced the target-only divergence. Replaying frozen proposals also reproduced the speculative divergence without live NPU draft inference. The latter depended on earlier target execution history; the controls found no position drift or specific rollback-boundary bug. The responsible floating-point or backend operation remains unidentified.

Under this frozen tested target stack, UB2 did not preserve the strict direct-ID output contract. Only the target reference remained in the formal registry, leaving no qualified intervention pair to evaluate. No formal R2 performance run occurred, and the follow-up diagnostics are invalid for performance claims. Qualification excluded an intervention motivated by the measured bottlenecks before its speed was evaluated.

7.2 A prospective cost guard on M4

The M4 experiment asks whether a frozen rule based on already-realized complete cost can control continuation while preserving exact output and profitable execution against target-only. It does not assume that adding control improves on fixed speculation. We retain the Qwen3-8B/0.6B Q8_0 pair on llama.cpp Metal, fixed $k = 2$, greedy generation, and a 128-token cap. Guard begins on the same speculative path as Always-Spec and can fall back permanently to target-only within a request. It changes neither proposal width nor device placement.

At each eligible completed round, Guard divides all elapsed charged generation cost by committed output progress C. This count includes the initial target seed and committed correction or bonus tokens, but excludes rejected proposals. Guard continues if this cumulative cost per token is at most $(1 - m)R_{\text{ref}}$, a target-only reference frozen from calibration; otherwise it falls back. If a decision is reached with zero committed progress, Guard also falls back. Be-

fore each control decision, invalid candidate suffix state is removed from the target and draft contexts, preserving committed history for target-only continuation. The enclosing post-prefill clock includes generation-loop initialization, seed-token sampling, drafting, target work, synchronization, state reconciliation, control, and transition costs. Model loading, sampler/speculator setup, initial prompt prefill, and the speculative begin hook precede the timer; the result is not whole-request latency.

Calibration freezes R_{ref} as the median of prompt-median target latency per output token. A preregistered grid selects decision round and margin by exactness and calibration GM latency relative to the two-fixed-mode oracle, with frozen tie-breaks. The selected rule has margin 0, first decides after 8 rounds, then after each later completed round. Early-ending requests make no decision. Online control receives no task/family labels, future length, request-specific target latency or oracle outcomes.

7.3 Held-out evaluation

Historical outcomes informed the design; prospective means policy freeze before reserved outcomes. Exposure exclusion and deterministic selection give disjoint calibration, qualification and held-out splits of 40, 16 and 60 prompts, balanced across HumanEval and Spec-Bench. Policy and reference precede qualification/formal measurement; the appendix retains the audited freeze-hash metadata defect. Always-Spec and Guard each qualify 48/48 direct-ID pairs.

The prospective M4 study runs 3 repetitions of target-only, Always-Spec, and Guard on each held-out prompt, randomizing condition order within prompt and repetition: 540 observations. Both speculative conditions preserve 180/180 complete direct-ID sequences, with zero mismatches. For each prompt and condition, analysis takes the median latency across repetitions, then the geometric mean of prompt latency ratios. This ratio of condition medians differs from the historical M4 text-path hardware estimator. Confidence intervals use 10,000 prompt bootstrap resamples with seed 20260924; repetitions are not independent sampling units.

Guard is profitable against target-only on this held-out workload (Table 5). Fixed speculation has the higher aggregate speedup point estimate. A reviewer-requested post-study paired secondary analysis of the frozen prompt medians gives Guard/Always-Spec latency 1.0271 [1.0130, 1.0436]. The interval is above parity: Guard had higher latency on this comparison. By prompt median, Guard is faster on 24 prompts, Always-Spec on 36, with 0 ties. Guard falls back in 30.6% of its 180 request executions; this measures control activity, not the fraction of beneficial decisions. Guard's geometric-mean latency relative to the offline oracle is 1.0278×. That oracle chooses the lower target-only or Always-Spec median after measurement and was unavailable to the runtime. It is a descriptive hindsight comparator, not an achievable-policy guarantee.

Table 5. M4 ratios of paired prompt medians. Both speculative policies preserve 180/180 direct-ID sequences. Speedup and relative latency have opposite favorable directions. The oracle selects the lower of two measured fixed-mode medians with hindsight. Guard/Always is post-study secondary; values above one mean Guard is slower.

Ratio (×)	GM	95% prompt CI
Speedup: above one favors speculation		
Target / Guard	1.3114	[1.2487, 1.3787]
Target / Always-Spec	1.3469	[1.2867, 1.4088]
Relative latency: lower is better		
Guard / offline oracle	1.0278	[1.0137, 1.0446]
Post-study paired secondary latency ratio		
Guard / Always-Spec	1.0271	[1.0130, 1.0436]

A post-study reconstruction of all original medians finds target-only faster than Always on only 2/60 prompts, both Spec-Bench. Always/oracle is 1.0007×: little opportunity for choosing between these two measured fixed modes. Guard falls back in all six executions of those two prompts but still loses to target-only on their medians. Guard also remains slower in a family-stratified paired bootstrap sensitivity analysis; Appendix P reports this separate new interval. These are hindsight comparisons, not transition-cost estimates.

Guard preserves IDs but supplies no incremental benefit over Always-Spec. Fallback activity does not establish beneficial decisions or risk reduction. No runtime energy claim is made. Policy transfer and stable small-stratum effects remain unestablished; the archived policy-freeze defect and original stopped pipeline markers are documented separately.

8 Related Work

Performance and heterogeneous execution. Speculative decoding amortizes target computation under its algorithmic assumptions (Leviathan et al., 2023). Prior work already documents verification and workload limits: Liu et al. (2026b) study verification dominance and batch effects in vLLM; Kim et al. (2026) account for conditional execution cost and runtime control; Lyu et al. (2026) derive edge–cloud profitability conditions. On consumer Apple silicon, Chordiya (2026) report gains and slowdowns across five backend configurations, diagnose ineffective batch-parallel verification in one quantized Metal path, test output agreement, and observe target-only batch-shape argmax changes. Slowdown and correctness gating have already been studied,

as has heterogeneous placement: Dovetail, DuoDecoding, and Mirror-SD study GPU/CPU or GPU/NPU paths (Zhang et al., 2025; Lv et al., 2025; Bhendawade et al., 2025); Ruiz y Mesa et al. (2026) combine an analytical cost model with compiler-oriented CPU/GPU partitioning, and Lever jointly optimizes draft, verification, and CPU/NPU execution for a smartphone target (Wang et al., 2026). State reuse has prior art in compressed draft KV memory (Yuan et al., 2026). Our 358H study measures one interface that combines replay and proposal; it establishes no general placement or persistence result.

Numerical paths and output contracts. Zhang et al. (2026) distinguish ragged-batch synchronization errors from residual numerical mismatches. Yuan et al. (2025) study output variation across system configurations, and LLM-42 addresses deterministic inference through verified speculation (Gond et al., 2026). Numerical path dependence is not new here. We prospectively qualify specific RTX/B570 paths against captured generated IDs, separately from historical text-path timing. B570 selects H1 on exposed discovery, then freezes full predictions before 72 reserved live runs; four non-reference trajectories rule out simply echoing the reference. Separate forks test sampled intervention effects. RTX V4 contrasts target-only UB2 with prospective proposal-conditioned replay on the same cohort and matches three-path qualification/cost. These tests add finite predictive discrimination to known path sensitivity, without a new numerical phenomenon, independent numerical implementation or unique kernel cause.

Adaptive speculation and the study boundary. TurboSpec adjusts speculation for serving goodput (Liu et al., 2025); BanditSpec selects configurations online (Hou et al., 2025); ACL HeteroSpec adapts verification from contextual entropy (Liu et al., 2026a). Adaptive control is established prior work. Our M4 Guard is a qualified runtime control test, not a new policy advantage: it beats target-only, but fixed speculation is faster in the post-study paired secondary comparison. Our measurements test prospective direct-ID qualification, frozen trajectory prediction, sampled intervention effects and realized cost on qualified paths.

9 Limitations

Narrow Qwen3 models, greedy caps and jointly varying hardware/backend/OS/quantization limit inference: no hardware ranking, population failure rate, functional pass@k or production claim follows. Strict-ID compatibility is reference-conditional; historical text-path timing remains distinct. RTX V4 compares all three paths within one cost study, but batching is blocked. Its timer includes logging and supplies no qualified batched speedup, deployment estimate, throughput or energy result.

B570 H1 has 24 reserved prompts and four reference-divergent trajectories on one stack. The fork samples one checkpoint per new run; persistent UB128 continuation is not a single-block restoration or an explanation of the old four tasks. RTX V4's 19 prompts are all remaining exposure-audited tasks, not a representative population or an independent B cost cohort. Its UB2 failure rejects only that candidate. Replay uses drafting and shared numerical routines; necessity, minimality and uniqueness of proposal/history remain unknown. Native micro-batch settings also affect draft, so B compares whole-path configurations. No unique operation or portable numerical cause is identified.

Intervals exclude session/recalibration variation; events/repeats are correlated. Exposure checks cover retained records. Hashes identify bytes; reported host/server order, Git and anonymous ordinals do not authenticate original absolute time or omitted binaries. 358H runtime raw traces and exact old sequence ZIP are unavailable; preserved arrays and processed cost records remain independently checkable. Its timing region combines reconstruction/replay with proposal, so their separate costs are unavailable. The zero-draft bound holds only at fixed measured progress and target work.

M4 Guard/Always, family sensitivity and fixed-mode opportunity are post-study secondary. The oracle chooses two measured modes with hindsight, not arbitrary switching policies. The M4 metadata defect retains original error and stopped markers. Runtime energy and 358H persistence are unmeasured; energy domains differ; M4's corrected synthetic ceiling was unexecuted.

10 Conclusion

On the tested B570 stack, frozen draft-free UB1 predictions match 72/72 reserved capped sequences, including four reference-divergent prompts; reference/live is 60/72 and remains blocked. Separate forks change scores without sampled immediate decision changes. RTX V4 discriminates predefined candidates on the same 19 prompts: target-only UB2 matches 51/57, replay 57/57. This does not prove history necessary. Matched cost blocks batching by IDs; qualified serialization is slower within its instrumented boundary. Fixed-progress 358H target work prevents parity at zero draft/residual cost, and M4 Guard loses to fixed speculation. Each conclusion is limited to its tested reference and path; no portable mechanism or optimization is established.

REFERENCES

Bhendawade, N., Nishu, K., Kundu, A., Bartels, C., Cho, M., and Belousova, I. Mirror speculative decoding: Breaking the serial barrier in LLM inference. *arXiv preprint arXiv:2510.13161v2*, 2025. URL https://arxiv.org/abs/2510.13161v2.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Petroski Such, F., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Hebgen Guss, W., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374v2*, 2021. URL https://arxiv.org/abs/2107.03374v2.

Chordiya, P. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware. *arXiv preprint arXiv:2607.17283*, 2026. doi: 10.48550/arXiv.2607.17283. URL https://arxiv.org/abs/2607.17283.

Gond, R., Kamath, A. K., Ramjee, R., and Panwar, A. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint arXiv:2601.17768v2*, 2026. URL https://arxiv.org/abs/2601.17768v2.

Hou, Y., Zhang, F., Du, C., Zhang, X., Pan, J., Pang, T., Du, C., Tan, V., and Yang, Z. BanditSpec: Adaptive speculative decoding via bandit algorithms. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 24045–24079. PMLR, 2025. URL https://proceedings.mlr.press/v267/hou25h.html.

Kim, S., Kim, J., Cho, Y., and Seo, J. Runtime-controlled evaluation of speculative decoding methods. *Electronics*, 15(14):3179, 2026. doi: 10.3390/electronics15143179. URL https://www.mdpi.com/2079-9292/15/14/3179.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, S., Ye, Y., Zhu, Q., Cao, Z., and He, Y. HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12930–12947. Association for Computational Linguistics, July 2026a. doi: 10.18653/v1/2026.acl-long.589. URL https://aclanthology.org/2026.acl-long.589/.

Liu, X., Park, J., Hu, L., Kwon, W., Li, Z., Zhang, C., Du, K., Mo, X., You, K., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput. *arXiv preprint arXiv:2406.14066v3*, 2025. URL https://arxiv.org/abs/2406.14066v3.

Liu, X., Yu, J., Park, J., Stoica, I., and Cheung, A. Speculative decoding: Performance or illusion? In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1719–1741, 2026b. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html.

Lv, K., Guo, H., Guo, Q., and Qiu, X. DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting. *arXiv preprint arXiv:2503.00784*, 2025. URL https://arxiv.org/abs/2503.00784.

Lyu, Y., Irukulapati, B., and Champati, J. P. Speculation at a distance: Where edge-cloud speculative decoding actually pays off. *arXiv preprint arXiv:2606.25091v2*, 2026. URL https://arxiv.org/abs/2606.25091v2.

Ruiz y Mesa, A., Korol, G., Riesterer, M., Cardoso de Lima, J. P., and Castrillon, J. Compiler-assisted speculative sampling for accelerated LLM inference on heterogeneous edge devices. In *8th Workshop on Accelerated Machine Learning (AccML), co-located with HiPEAC*, January 2026. URL https://accml.dcs.gla.ac.uk/papers/2026/8th_AccML_paper_8.pdf.

Wang, T., Li, F., Sun, Y., Gao, W., and Ren, J. Lever: Speculative LLM inference on smartphones. *arXiv preprint arXiv:2605.16786*, 2026. doi: 10.48550/arXiv.2605.16786. URL https://arxiv.org/abs/2605.16786.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.456. URL https:

//aclanthology.org/2024.findings-acl.456/.

Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint arXiv:2506.09501v2*, 2025. URL https://arxiv.org/abs/2506.09501v2.

Yuan, T., Liao, C., and Wen, Z. Strong drafts need compact memories: Long-context speculative decoding with compressed KV cache. *arXiv preprint arXiv:2608.30252*, 2026. URL https://arxiv.org/abs/2608.30252. EMNLP 2026 Findings.

Zhang, L., Zhang, Z., Xu, B., Li, R., Tian, Z., Mei, S., and Li, D. Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference. *arXiv preprint arXiv:2412.18934v2*, 2025. URL https://arxiv.org/abs/2412.18934v2.

Zhang, R. H., Dey, S., Mishra, A., Wu, H., Li, B., and Zhang, R. Correctness forensics for batch speculative decoding: Diagnosing the ragged tensor problem. *arXiv preprint arXiv:2510.22876v4*, 2026. URL https://arxiv.org/abs/2510.22876v4. Accepted to Findings of EMNLP 2026.